

Algorithmic Differentiation

Application to Optimisation and Machine Learning

Short Course (Day 1)

Pavanakumar Mohanamurthy

Excerpts from “Numerical Optimisation” course by Dr. Jens D Mueller, Queen Mary University of London. *(notes and figures with permission)*

Hands-on Practice

Time to apply what we've learned!

Using PyTorch's `torch.autograd`
for real optimization problems

Python Basics: Variables and Data Types

Python is dynamically typed - no need to declare types

```
# Basic data types
x = 5                # Integer
y = 3.14            # Float
name = "AutoDiff"   # String

# Lists (mutable, ordered)
numbers = [1, 2, 3, 4, 5]

# Dictionaries (key-value pairs)
config = {"lr": 0.01, "epochs": 100}
```

Key point: Python uses indentation for code blocks!

Python Basics: Functions and Loops

Functions are defined with def keyword

```
# Define a function
def rosenbrock(x, y):
    return (1 - x)**2 + 100*(y - x**2)**2

result = rosenbrock(0.0, 0.0) # Call it
```

Loops and list comprehensions

```
# For loop
for i in range(5):
    print(f"Iteration {i}")

# List comprehension (Pythonic way!)
squares = [x**2 for x in range(10)]
```

NumPy: The Foundation of Scientific Python

NumPy provides efficient multi-dimensional arrays

```
import numpy as np

# Create arrays
x = np.array([1, 2, 3, 4, 5])      # 1D array
A = np.array([[1, 2], [3, 4]])    # 2D array
zeros = np.zeros((3, 3))          # 3x3 zeros
random = np.random.randn(2, 3)    # Random normal

# Array operations (element-wise!)
y = x * 2                          # Multiply by 2
z = x + y                          # Addition
w = np.sin(x)                      # Sine
```

Why NumPy? Fast, vectorized operations (C backend)

NumPy: Mathematical Operations

Broadcasting and linear algebra

```
import numpy as np

x = np.array([1, 2, 3])
y = x + 10                # Broadcasting

# Matrix operations
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
C = A @ B                 # Matrix multiply
det = np.linalg.det(A)    # Determinant
norm = np.linalg.norm(x)  # L2 norm
```

Introduction to PyTorch

PyTorch: Deep Learning + Automatic Differentiation

Why PyTorch for this course?

- ▶ **Tensors:** Like NumPy arrays, but GPU-accelerated
- ▶ **Autograd:** Automatic differentiation engine
- ▶ **Dynamic:** Build computational graphs on-the-fly
- ▶ **Pythonic:** Easy to learn and debug

Think of it as: NumPy + Automatic Differentiation + GPU

PyTorch: Tensors

Tensors are the fundamental data structure

```
import torch

# Create tensors
x = torch.tensor([1.0, 2.0, 3.0])
y = torch.zeros(3, 3)
z = torch.randn(2, 4)

# Convert from NumPy
import numpy as np
tensor = torch.from_numpy(np.array([1, 2, 3]))

# Operations
b = x * 2
```


PyTorch: Automatic Differentiation

The magic: `requires_grad=True`

```
import torch

x = torch.tensor(2.0, requires_grad=True)
y = x**2 + 3*x + 1

y.backward()  # Compute gradients!
print(x.grad) # 2*x + 3 = 7.0
```

Key: `requires_grad=True` tracks operations, `.backward()` computes gradients, `.grad` accesses them

PyTorch: Multivariate Gradients

Computing gradients with respect to vectors

```
import torch

x = torch.tensor([1.0, 2.0], requires_grad=True)

# Rosenbrock function
f = (1 - x[0])**2 + 100*(x[1] - x[0]**2)**2

f.backward()
print(x.grad) # [df/dx0, df/dx1]
```

Remember: Gradients accumulate, use `x.grad.zero_()` to reset

PyTorch: Optimization Loop

Gradient descent with autograd

```
import torch

x = torch.tensor([0.0, 0.0], requires_grad=True)

for i in range(1000):
    f = (1 - x[0])**2 + 100*(x[1] - x[0]**2)**2
    f.backward()
    with torch.no_grad():
        x -= 0.001 * x.grad
    x.grad.zero_()
```

Immutable Programming: Why It Matters for AD

PyTorch tracks operations to build a computational graph

Problem: In-place operations can break gradient computation!

```
x = torch.tensor([1.0, 2.0], requires_grad=True)
y = x * 2
x[0] = 5.0      # In-place modification - BREAKS graph!
y.backward()    # Error: modified by in-place operation
```

Why? The graph stores references to tensors. Modifying them invalidates the recorded history needed for backpropagation.

Immutable Programming: Best Practices

Create new tensors instead of modifying existing ones

```
# BAD: In-place operations  
x += 1           # Modifies x in-place  
x.add_(1)        # Underscore = in-place  
  
# GOOD: Create new tensors  
x = x + 1        # Creates new tensor  
x = x.add(1)     # Returns new tensor
```

Exception: Use `torch.no_grad()` for parameter updates

```
with torch.no_grad():  
    x -= lr * x.grad # Safe: not tracked  
x.grad.zero_()      # Reset gradients
```

How PyTorch Detects In-place Violations

PyTorch uses a version counter mechanism (lazy detection)

```
x = torch.tensor([1.0, 2.0], requires_grad=True)
y = x * 2

print(x._version)    # 0
x[0] = 5.0           # In-place modification
print(x._version)    # 1 (incremented!)

y.backward(torch.ones(2)) # RuntimeError here!
```

How it works:

- ▶ Each tensor has a `_version` counter
- ▶ Operations record the version when added to the tape
- ▶ During `backward()`, PyTorch checks if versions match
- ▶ Mismatch → `RuntimeError`

Hands-on Exercise 1: Rosenbrock Optimization

Run the code and observe:

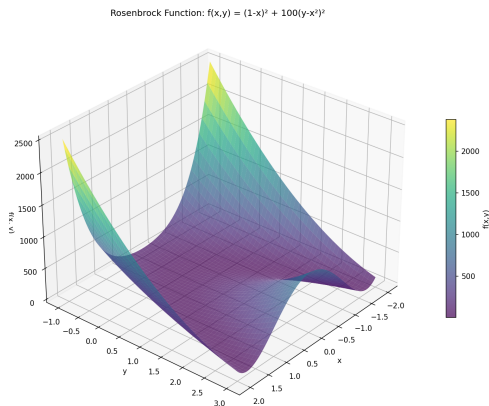
- ▶ CMA-ES vs L-BFGS optimization
- ▶ Analytical = Autograd gradients (exact!)
- ▶ FD error vs step size h

Exercise (Section 6): Implement central difference:

$$\frac{\partial f}{\partial x} \approx \frac{f(x+h) - f(x-h)}{2h}$$

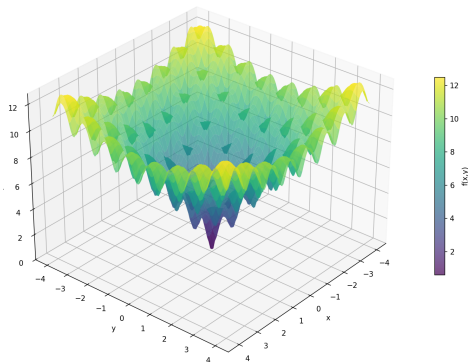
Questions: Why does FD error increase for very small h ?

File: `nathan/moss.py`



Hands-on Exercise 2: Ackley Function — Local vs Global Minima

Ackley Function: Many Local Minima, One Global Minimum at (0,0)



Key insight: Many local minima (bumps)!

Run the code and observe:

- ▶ CMA-ES finds global minimum at (0,0)
- ▶ L-BFGS gets **stuck** in local minimum!

Exercise: Try different starting points
→ When does L-BFGS succeed/fail?

Questions: What is the “basin of attraction”?

File: `python/ackley.py`

Hands-on Exercise 3: Bézier Curves and Arc Length

Arc length:

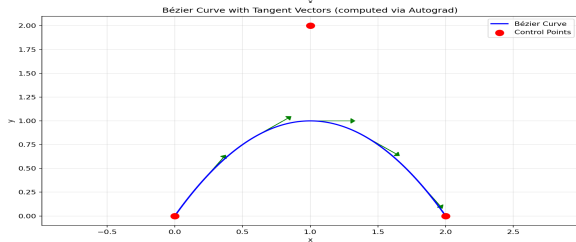
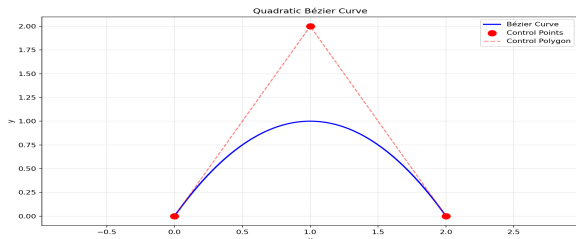
$$L = \int_0^1 \sqrt{\left(\frac{dx}{dt}\right)^2 + \left(\frac{dy}{dt}\right)^2} dt$$

Observe:

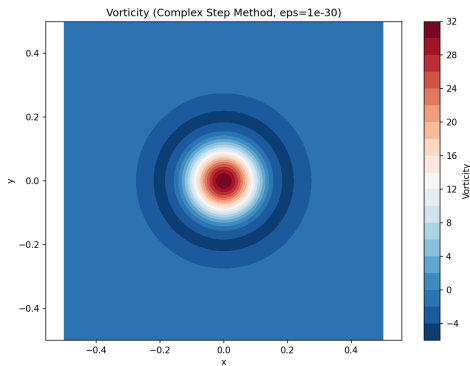
- ▶ Autograd computes $\frac{dx}{dt}, \frac{dy}{dt}$
- ▶ Tangent vectors on curve

Exercise: Create a **cubic** Bézier curve (4 control points)

File: `python/cad.py`



Hands-on Exercise 4: Complex Step Differentiation



Complex step trick:

$$f'(x) \approx \frac{\text{Im}[f(x + ih)]}{h}$$

No cancellation errors! Use $h = 10^{-100}$

Run the code and observe:

- ▶ Complex step: stable for *any* h
- ▶ Finite diff: breaks for small h

Exercise: Implement the **analytical** vorticity formula and compare

File: `python/vortex.py`